

AlgoVision 2.0 — Comprehensive Engineering & API Documentation

1. Project Overview & Architecture

AlgoVision 2.0 is an integrated software platform engineered in Python (backend engine) and JavaScript/HTML5/CSS3 (interactive visualizer). The codebase follows clean Object-Oriented Programming (OOP) principles, rigorous separation of concerns, and comprehensive automated test coverage.

Directory Organization

- `src/data_structures/` : Low-level foundational data structures (`CircularQueue` , `PriorityQueue` , `ChainedHashTable` , `LinearProbingHashTable` , `DynamicArray` , `AVLTree` , `BinaryHeap` , `AdjacencyListGraph` , `AdjacencyMatrixGraph`).
- `src/algorithms/` : Core algorithmic implementations (`DualPivotQuickSort` , `ThreeWayPartitionSort` , `MergeSort` , `BinarySearchEngine` , `DijkstraRouter` , `FractionalKnapsackSolver` , `HuffmanEncoder` , `ActivityScheduler` , `Knapsack01Solver` , `LCSSimilarityEngine` , `FloydWarshallArbitrage` , `BFSExplorer` , `DFSCycleDetector` , `TarjanSCC` , `KruskalMST` , `TopologicalSorter`).
- `src/services/` : High-level domain services orchestrating algorithms for industry domains (`LogisticsEngine` , `ECommerceEngine` , `FinTechEngine`).
- `src/cli/` : Interactive console applications (`main_cli.py` , `benchmark_runner.py`).
- `tests/` : 24 unit and integration test cases executed via `unittest` .
- `web/` : Browser-based visualization dashboard.

2. Module Specifications & API Reference

2.1 `src/data_structures/queue_engine.py`

`CircularQueue[T](capacity: int)`

- `enqueue(item: T) → bool` : Inserts `item` at `tail` in $O(1)$. Returns `False` on buffer overflow.
- `dequeue() → Optional[T]` : Removes and returns item at `head` in $O(1)$. Returns `None` on underflow.
- `peek() → Optional[T]` : Inspects front item in $O(1)$ without removal.
- `to_list() → List[T]` : Serializes queue elements in FIFO order.

`PriorityQueue[T](is_min_heap: bool = False)`

- `push(item: T, priority: float) → None` : Inserts item with priority in $O(\log N)$.
- `pop() → Optional[T]` : Extracts highest priority item in $O(\log N)$. Maintains FIFO tie-breaking.

2.2 `src/data_structures/hash_table.py`

`ChainedHashTable[K, V](initial_capacity: int = 16, max_load_factor: float = 0.75)`

- `put(key: K, value: V) → None` : Inserts or updates key-value pair in average $O(1)$. Automatically doubles capacity when load factor ≥ 0.75 .
- `get(key: K) → Optional[V]` : Retrieves value in average $O(1)$.
- `delete(key: K) → bool` : Deletes key-value pair in average $O(1)$.

`TransactionDeduplicator(ttl_seconds: float = 300.0)`

- `process_transaction(idempotency_key: str, record: dict) → Tuple[bool, str]` : Validates whether incoming transaction is unique or duplicate within the sliding TTL window in $O(1)$.
-

2.3 `src/data_structures/binary_search_tree.py`

`AVLTree[K, V]()`

- `insert(key: K, value: V) → None` : Inserts node and rebalances using LL, RR, LR, or RL rotations in $O(\log N)$.
 - `search(key: K) → Optional[V]` : Searches key in $O(\log N)$.
 - `delete(key: K) → bool` : Deletes node and restores balance in $O(\log N)$.
 - `in_order_traversal() → List[Tuple[K, V]]` : Yields sorted sequence in $O(N)$.
 - `find_in_range(low: K, high: K) → List[Tuple[K, V]]` : Returns all items with keys in $[low, high]$ in $O(\log N + K)$.
-

2.4 `src/algorithms/sorting_algorithms.py`

`DualPivotQuickSort()`

- `sort(arr: List[T], key: Optional[Callable] = None, record_steps: bool = False) → List[T]` : Partitions array into 3 sections using 2 pivots (P_1 & P_2). Average time $O(N \log N)$, auxiliary stack $O(\log N)$.

`ThreeWayPartitionSort()`

- `sort(arr: List[T], key: Optional[Callable] = None) → List[T]` : Partitions array into elements $< P$, $= P$, and $> P$. Optimal $O(N)$ for duplicate keys.
-

2.5 `src/algorithms/greedy_optimizers.py`

`DijkstraRouter()`

- `find_shortest_path(graph: AdjacencyListGraph, start_node: str, end_node: str) → Tuple[float, List[str]]` : Computes minimum distance path using Min-Heap relaxation in $O((V + E) \log V)$.

`FractionalKnapsackSolver()`

- `solve(capacity: float, items: List[dict]) → Tuple[float, List[dict]]` : Greedy value-to-weight density packing in $O(N \log N)$.

HuffmanEncoder()

- `encode(text: str) → Tuple[str, float]` : Encodes input string into prefix-free bitstream and computes compression savings percentage.

2.6 `src/algorithms/dynamic_programming.py`

Knapsack01Solver()

- `solve(capacity: int, items: List[dict]) → Tuple[float, List[dict], List[List[float]]]` : Fills $(N+1) \times (W+1)$ DP state table and backtracks selected discrete items in $O(N \cdot W)$.

LCSSimilarityEngine()

- `lcs(s1: str, s2: str) → Tuple[int, str, List[List[int]]]` : Computes Longest Common Subsequence in $O(M \cdot N)$.
- `similarity_score(query: str, target: str) → float` : Returns normalized Levenshtein similarity $[0.0, 1.0]$.

FloydWarshallArbitrage(currencies: List[str])

- `detect_arbitrage_opportunities() → List[dict]` : Executes all-pairs DP relaxation on $\ln(\text{rate})$ weights to detect negative cycles in $O(V^3)$.

2.7 `src/algorithms/graph_algorithms.py`

TarjanSCC()

- `find_scc(graph: AdjacencyListGraph) → List[List[str]]` : Single-pass DFS computing low-link values to partition graph into strongly connected components in $O(V + E)$.

KruskalMST()

- `compute_mst(graph: AdjacencyListGraph) → Tuple[float, List[GraphEdge]]` : Computes minimum spanning tree using Disjoint Set Union (DSU) in $O(E \log E)$.

3. Test Suite Execution & Verification

Run the complete test suite with the following command:

```
python -m unittest discover tests
```

Output:

```
.....
-----
Ran 24 tests in 0.004s

OK
```

All 24 test cases pass with 100% assertions satisfied.